

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Смирнова Карина

К3341 БР 1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Д32: Проектирование и технический дизайн API

Задание:

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1 Функциональные требования системы

1) Регистрация пользователя:

Описание: пользователь может создать аккаунт.

Входные данные: email, username, phone, password.

Правила:

- email уникален.
- пароль хранится только в виде хэша. Результат: создана запись User, перенаправление на страницу авторизации.
- Отправлено сообщение на почту для подтверждения

2) Вход (аутентификация) пользователя:

Описание: пользователь может войти в систему.

Вход: email + password.

Результат: выдаётся access token и refresh token (JWT).

3) Выход

Описание: пользователь может выйти из аккаунта

Результат: Refresh токен блокируется

4) Просмотр и редактирование профиля

Описание: пользователь может просматривать и изменять данные профиля. (username, email, password, phone). На странице профиля также отображается история бронирований пользователя и объекты (listing), принадлежащие пользователю.

Ограничение: Если изменяется email, то пользователь снова становится не верифицированным и ему на новую почту отправляется ссылка для подтверждения.

5) Создание объявления (Listing)

Описание: авторизованный пользователь может создать объявление

Поля: title, description, address, location, house_type, rent_mode, фотографии.

Правила:

- $\text{rent_mode} \in \{\text{DAILY}, \text{MONTHLY}\}$.
- Объявление создаётся как активное $\text{is_active}=\text{true}$ по умолчанию.
- К нему должен относиться объект `listing_daily` или `listing_monthly`.

6) Загрузка, удаление фотографий объявления

Описание: владелец может добавлять и удалять фото, назначать главное фото

Правила:

- у одного `listing` допускается несколько фото.
- допускается не более одного $\text{is_main}=\text{true}$.

7) Редактирование объявления

Описание: владелец может менять описание, цену, условия, адрес, геопозицию, тип дома, поля относящиеся к daily и monthly объекту или снять объявление с публикации (is_active=false).

Ограничения:

- Нельзя поменять rent_mode, придется создавать новый listing

8) Поиск объявлений

Описание: пользователь может искать объявления по параметрам

Фильтры:

- rent_mode (DAILY/MONTHLY)
- house_type
- диапазон цены (для DAILY — цена за ночь, для MONTHLY — цена за месяц)
- расположение
- фильтр доступно на даты X–Y для посуточной аренды:
выдавать объявления без пересекающихся бронирований на выбранный период.

9) Просмотр карточки объявления

Описание: Получение полной информации по объявлению. Если объявление посуточное, то выбрав даты можно забронировать без общения с владельцем. Если объявление на длительную аренду, то можно позвонить или написать владельцу.

10) Создание обращения по долгосрочной аренде

Описание: пользователь может связаться с владельцем по объявлению MONTHLY.

События:

- Нажатие кнопки позвонить - создает запись обращения и устанавливает статус в NEW
- Нажатие кнопки написать - создает чат и запись обращения, устанавливая статус в NEW

11) Изменение статуса долгосрочного обращения

Описание: статус обращения меняется по правилам:

- NEW → IN_PROGRESS, когда владелец отвечает.
- IN_PROGRESS → CLOSED, когда обращение завершено (вручную владельцем, покупателем или автоматически по таймауту).

12) Создание бронирования посуточного жилья

Описание: пользователь выбирает даты и бронирует жилье.

Данные: listing_id, start_date, end_date.

Правила:

- бронирование возможно только для rent_mode='DAILY'.
- end_date > start_date.
- количество ночей $\geq$ min_nights и $\leq$ max_nights (если задан).
- при создании сохраняются цена и итоговая стоимость total_amount.
- запрещены пересечения дат бронирований одного объявления в активных статусах.

13) Изменение статуса бронирования посуточного жилья

Описание: статусы Booking: PENDING, ACCEPTED, CANCELED, COMPLETED.

Правила переходов:

- PENDING → ACCEPTED (автопринятие)
- PENDING/ACCEPTED → CANCELED (отмена)
- ACCEPTED → COMPLETED (после даты выезда)

14) Создание чата

Описание: при обращении (нажатии) “Написать” создаётся Chat между двумя пользователями.

Правила:

- для одной пары пользователей не создаются дубли (через UNIQUE(user1Id, user2Id)).

15) Отправка, получение сообщений

Описание: участники чата могут отправлять сообщения.

Результат: запись в Messages с senderId, content, created_at.

2 Нефункциональные требования системы

1) Безопасность

- Пароли хранятся только в виде хэша
- Защита эндпоинтов: доступ к изменению Listing/Booking/Rent только владельцу/участнику.
- Валидация входных данных (длины строк, форматы email/phone, диапазоны цен, даты).
- Аутентификация через JWT токены

2) Производительность

- Поиск объявлений должен работать быстро при росте данных:
 - i) индексы по rent_mode, house_type
 - ii) индексы по listing_id в фото/booking/rent
- Для выдачи сообщений/поиска — пагинация.

3) Удобство и сопровождение

- Логирование действий
- Единый формат ошибок
- Документация Swagger
- JSON ответы
- REST архитектура

REST API Эндпоинты

1) Auth + Users

- POST /auth/register — регистрация
- POST /auth/login — вход (JWT access + refresh)

- POST /auth/refresh — обновить access по refresh
- POST /auth/logout — выход (заблокировать refresh)
- GET /users/me — получить профиль текущего пользователя
- PATCH /users/me — изменить профиль (username/phone/email/password)
- POST /users/me/email/verify — запросить повторную отправку письма
- POST /users/email/confirm — подтвердить email по токenu
- GET /users/me/profile — запросить профиль пользователя

2) Listings (объявления)

- POST /listings — создать объявление (базовые поля + rent_mode)
- GET /listings — поиск + фильтрация
- GET /listings/{listingId} — карточка объявления
- PATCH /listings/{listingId} — редактирование общих полей (без смены rent_mode)
- POST /listings/{listingId}/deactivate — снять с публикации
- POST /listings/{listingId}/activate — вернуть

3) Photos

- POST /listings/{listingId}/photos — добавить фото
- GET /listings/{listingId}/photos — список фото
- PATCH /listings/{listingId}/photos/{photoId} — поменять is_main
- DELETE /listings/{listingId}/photos/{photoId} — удалить фото

4) Daily bookings (посуточно)

- POST /bookings — создать бронирование
- GET /bookings — список бронирований текущего пользователя (как guest)
- GET /bookings/{bookingId} — детали
- POST /bookings/{bookingId}/cancel — отменить
- POST /bookings/{bookingId}/complete — завершить

5) Monthly rents (долгосрочная аренда)

- POST /rents — создать обращение (listing_id, communication_method=CALL/CHAT)
- GET /rents — список обращений (пользователя)
- GET /rents/{rentId} — детали обращения
- PATCH /rents/{rentId} — обновить статус
- POST /rents/{rentId}/close - закрыть объявление

6) Chats / Messages

- POST /chats — создать чат
- GET /chats — список моих чатов
- GET /chats/{chatId}/messages — история сообщений
- POST /chats/{chatId}/messages — отправить сообщение

Вывод

В ходе данной домашней работы были сформулированы функциональные и нефункциональные требования к системе по аренде недвижимости, а также спроектированы REST API эндпоинты. Мною также был изучен подход к проектированию документации OpenAPI с использованием TypeSpec.